CodeMySpec Google Ads API Design Document

CodeMySpec: Google Ads API Integration

Design Document for Google Ads API Access Application

Company: CodeMySpec **Contact:** johns10@gmail.com **Document version:** 1.1 **Date:** June 22, 2026 **Requested access level:** Basic access (with a roadmap to Standard access)

---

1. Executive Summary

CodeMySpec is a requirements-driven software development platform. It helps software teams plan, build, and operate products by coordinating AI agents (large language models) against a structured model of the work to be done. CodeMySpec runs on top of **Claude Code**, Anthropic's agent runtime.

This document describes a new capability: an **AI-assisted Google Ads management surface**, delivered as a **Model Context Protocol (MCP) server**. The MCP server exposes a small, fixed set of tools to the AI agent. Read tools return campaign performance data. Write tools modify the account (budgets, bids, keywords, ad copy, campaign status).

Human approval is built into the runtime, not bolted on. Claude Code prompts the human operator for permission before any tool call executes. Mutating tools are configured to always require explicit per-call approval, so **no change reaches a live Google Ads account until the human approves that specific tool call**. The agent never manages spend autonomously.

This document covers two deployment contexts:

1. **Internal use**: CodeMySpec manages advertising for its own product.
2. **Customer-facing use**: CodeMySpec customers connect their own Google Ads accounts so the agent can help them manage their advertising. This is the primary review target and drives the OAuth and data-isolation requirements below.

---

2. Company & Product Overview

CodeMySpec is a development harness that treats every artifact a software project needs (specifications, tests, implementations, reviews) as nodes in a requirement graph. AI agents work through that graph one step at a time, and a human operator approves the agent's actions as it goes.

The product is built on Claude Code, which provides a **permission system around every tool call**: the agent can read freely, but actions the operator deems sensitive are gated behind an approval prompt. This Google Ads integration uses exactly that mechanism (reads run automatically, writes require approval) rather than inventing parallel approval machinery.

---

3. Goals and Non-Goals

3.1 Goals

- Let an AI agent retrieve Google Ads performance data and surface insights.
- Let the agent draft specific, well-reasoned change proposals.
- Gate every mutating tool call behind Claude Code's human-approval prompt.
- Support both a single internal account and many customer-owned accounts with strict per-account isolation.

3.2 Non-Goals

- **No fully autonomous spend changes.** Every mutating tool call requires the operator to approve it in Claude Code before it executes.
- **No bulk or unattended account creation.** The tool does not mass-create accounts, campaigns, or ads programmatically.
- **No data resale or cross-account aggregation.** One customer's Google Ads data is never exposed to another customer or used to train shared models.
- **No circumvention of Google Ads policies.** The tool surfaces and respects policy and approval status; it does not attempt to evade review.

---

4. Users and Use Cases

4.1 Primary users

- **Solo founders and small software teams** who run their own Google Ads but lack a dedicated marketing analyst. They want expert-level analysis and concrete recommendations without hiring an agency.
- **CodeMySpec's internal team**, managing the company's own advertising.

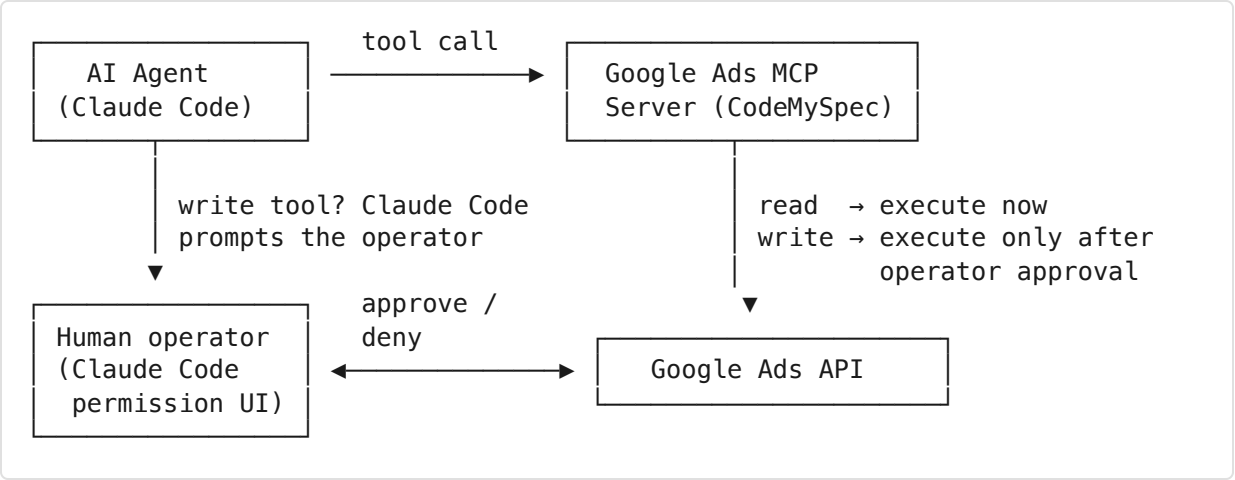
4.2 Representative use cases

1. **Performance review.** "How did my Search campaigns perform last 30 days?" The agent pulls campaign, ad group, and keyword metrics and summarizes spend, conversions, CPA, and trends. (Read only; runs automatically.)
2. **Budget reallocation.** The agent identifies an over-funded, under-converting campaign and proposes shifting budget. The agent calls `update_campaign_budget`; Claude Code prompts the operator to approve the change before it executes.
3. **Negative keyword cleanup.** The agent flags search terms wasting spend and calls `add_negative_keywords`. The operator approves the call before the keywords are added.
4. **Ad copy iteration.** The agent drafts new responsive search ad headlines and descriptions and calls `create_ad`. The operator approves; the new ad still goes through Google's own ad-approval process.
5. **Account health check.** The agent reports disapproved ads, limited-by- budget campaigns, and policy issues so the human can act. (Read only.)

---

5. Functional Design

5.1 Architecture overview



The MCP server is the only component that talks to the Google Ads API. It exposes a small, fixed set of tools. The human-in-the-loop gate is **Claude Code's built-in tool permission system**. Mutating tools are marked to always require approval, so the operator sees and confirms each write before it runs.

5.2 Tool surface (MCP tools)

Tool	Type	Approval	Google Ads API services used
`list_accessible_accounts`	Read	auto	CustomerService, GoogleAdsService
`get_account_overview`	Read	auto	GoogleAdsService (search)
`get_campaign_performance`	Read	auto	GoogleAdsService (search/searchStream)
`get_ad_group_performance`	Read	auto	GoogleAdsService (search)
`get_keyword_performance`	Read	auto	GoogleAdsService (search)
`get_search_terms_report`	Read	auto	GoogleAdsService (search)
`get_ad_performance`	Read	auto	GoogleAdsService (search)
`get_recommendations`	Read	auto	RecommendationService, GoogleAdsService
`update_campaign_budget`	Write	**prompt**	CampaignBudgetService
`update_bid`	Write	**prompt**	AdGroupService, AdGroupCriterionService
`add_keywords`	Write	**prompt**	AdGroupCriterionService
`add_negative_keywords`	Write	**prompt**	AdGroupCriterionService, CampaignCriterionService
`create_ad`	Write	**prompt**	AdGroupAdService
`update_campaign_status`	Write	**prompt**	CampaignService

Read tools execute immediately. Every write tool is configured so Claude Code prompts the operator before it runs. The tool arguments (target resource, new value) are shown in the approval prompt, so the operator sees exactly what will change before approving.

5.3 Approval flow

1. The agent decides on a change and calls a write tool (e.g. `update_campaign_budget`) with the target resource and new value.
2. Claude Code intercepts the call and shows the operator an approval prompt containing the tool name and its arguments.
3. The operator **approves** or **denies**.
4. On approval, the MCP server executes the corresponding Google Ads `mutate` operation and returns the result. On denial, no call is made to Google Ads.

This is the same permission mechanism that gates every other sensitive action in CodeMySpec, with no separate approval queue, dashboard, or custom audit service to build or operate.

5.4 Required Minimum Functionality (RMF) alignment

The integration is designed to satisfy Google's RMF expectations for a management tool:

- **Reporting**: performance reporting across campaign, ad group, keyword, search term, and ad levels.
- **Campaign management**: change budgets, bids, keywords, ad copy, and campaign status (all human-approved).
- **Account management**: list accessible accounts, surface recommendations, and report account health and policy issues.

---

6. Account Access & OAuth Model

6.1 Authentication

CodeMySpec uses **OAuth 2.0** with Google's standard consent flow, requesting the `https://www.googleapis.com/auth/adwords` scope. The application's developer token is held server-side and is never exposed to the AI agent or the browser.

6.2 Internal accounts

For CodeMySpec's own advertising, a single Google Ads account (or manager account) is linked once via OAuth. Refresh tokens are stored encrypted at rest.

6.3 Customer accounts (multi-tenant)

Each CodeMySpec customer connects **their own** Google Ads account through the OAuth consent screen. Key isolation properties:

- Each customer's OAuth refresh token is stored encrypted and scoped to that customer's CodeMySpec account.
- The MCP server resolves the correct customer credentials per request from the authenticated CodeMySpec session; an agent operating in one customer's context can never reach another customer's Google Ads data.
- Customers can disconnect at any time, which revokes the stored token.

- Where appropriate, customer accounts are linked under a Google Ads **manager account (MCC)** using the standard account-link consent flow.

6.4 Login customer / manager structure

When acting on a linked account, the MCP server sets the appropriate `login-customer-id` header and operates only on accounts the authenticated customer has explicitly linked and consented to.

---

7. Data Handling & Security

Concern	Approach
Credentials	OAuth refresh tokens encrypted at rest; developer token server-side only
Performance data	Fetched on demand; short-lived caching only, scoped per customer account
Cross-tenant isolation	Per-customer credential resolution; no shared data store across customers
Data retention	Customers can disconnect and request deletion of stored tokens and cached data
Secrets in logs	Structured logfmt logging; tokens and PII are never logged
Transport	All Google Ads API traffic over TLS

The AI agent receives only the data needed to answer the current request and never receives raw credentials. Google Ads data for one customer is never used to serve, train, or inform another customer's experience.

---

8. Compliance with Google Ads API Policies

- **Required Minimum Functionality**: reporting, campaign management, and account management as described in Section 5.4.
- **No policy circumvention**: the tool reads and surfaces ad approval and policy status; it never attempts to bypass review or detection.

- **Human accountability**: a specific human approval in Claude Code stands behind every mutating action, satisfying the expectation that a responsible party owns each change.
- **Rate limiting & quotas**: the server batches reads, respects API rate limits, and backs off on quota errors.
- **Terms & Conditions**: CodeMySpec agrees to and operates under the Google Ads API Terms and Conditions.

---

9. Scale & Quota Estimates

At launch (Basic access), expected usage is modest: a small number of internal and early-customer accounts, with operations dominated by read/reporting queries and a low volume of human-approved writes. Read traffic is batched and cached briefly to stay well within Basic access daily operation limits. As the customer base grows, CodeMySpec will apply for **Standard access** and provide updated volume projections at that time.

---

10. Operator Experience

The operator works in Claude Code. A typical session:

1. The operator asks the agent a question ("review my search campaigns") or gives a goal ("cut wasted spend").
2. The agent calls read tools and presents its analysis in the conversation.
3. When the agent proposes a change, it calls the corresponding write tool. Claude Code shows an approval prompt with the tool name and arguments (e.g. "update_campaign_budget: campaign X, $40/day → $25/day").
4. The operator approves or denies inline. Approved calls execute; denied calls do not.

No separate dashboard is required. The conversation and Claude Code's approval prompts are the interface.

---

11. Summary

CodeMySpec's Google Ads integration applies the platform's core discipline (**AI proposes, humans approve**) to advertising management, using Claude Code's built-in tool-permission system as the approval gate rather than custom machinery. It exposes the Google Ads API through a narrow, well-defined MCP tool surface, isolates customer data per tenant, and

ensures that no change reaches a live Google Ads account without explicit human authorization. We believe this design meets the Required Minimum Functionality and policy expectations for Google Ads API access and welcome the review team’s feedback.